MSBBSS04: Statistical programming in R

Lab: Collaboration & version control

Work preferably in pairs and perform all commands in this practical using RStudio and/or GitHub Desktop. At the end of this document you will find instructions about how and when to hand in your work.

Introduction

As a statistician / data scientist / developer you'll need to master many skills. Some of these skills are made simple with tools. This week we'll learn about a massively important tool for your tool kit: **version controlling**.

To document our activity and our changes in detail, we'll use Git. You can view Git as the ability to go back in time. Back to the very beginning of your project. A bonus: Git integrates nicely with RStudio.

In this lab we will learn:

1. How to integrate Git within our projects.
2. How to publish our projects as GitHub repositories.
3. How to go about collaboration with Git and GitHub.

Background

Git

Git is a free and open-source version control system for text files. It can handle extensive change logging for you, no matter the size of the project. Git is fast and efficient, but its effectiveness depends also on the frequency you instruct it to log your project's changes.

You can see Git as a blank canvas that starts at a certain point in time. Every time you (or others) instruct Git to log any changes that have been made, Git adds the changes that

are made to this canvas. We call the changes to the canvas `commits`. With every `commit` an extensive log is created that includes at least the following information:

- the changes made
- who made the changes
- metadata
- a small piece of text that describe the changes made

The difference between two commits - or the changes between them - are called `diffs`.

If you'd like to know much more about Git, this online book is a very good resource. If you'd like to practice with the command line interface use this webpage for a quick course. This book covers pretty much everything you need to marry Git and `R`.

GitHub

GitHub is a social and user interface to Git that allows you to work in repositories. These repositories can be seen as project folders in which you publish your work, but you can also use them as test sites for development, testing, etcetera. There is a distinction between private repositories (only for you and those you grant access) and public repositories.

Your public repositories can be viewed and forked (i.e., copied) by everyone on their own accounts. This allows them to work on a repository without affecting the master. You can also do this to your own repository, but then the process is called `branching` instead of forking. If you create an offline copy of a repository, the process is called `cloning`. The goals are also slightly different:

- **Forking**: continue development of a repository without altering the original repository
- **Branching**: create a *parallel world* within a repository to implement some specific change
- **Cloning**: develop a repository locally before syncing it with GitHub

GitHub's ability to branch, fork and clone is very useful as it allows other people and yourself to experiment on a repository before any definitive changes are `merged` with the `master`. If you're working in a forked repository, you can submit a `pull request` to the repository collaborators to accept or reject any suggested changes.

For now, this may be confusing, but I hope you recognize the benefits GitHub can have on the process of development and bug-fixing.

> **Installing R packages from GitHub**
>
> You can download certain GitHub repositories directly in R. For example, the most up-to-date version of the `mice` package in `R` can be directly installed from the `mice` repository with the following code:

```
pak::pak("stefvanbuuren/mice")
```

Note. this requires installing pak first.
Loading packages in `R` directly from their respective GitHub repositories allows you to obtain the latest - often improved and less buggy - iteration of that software even before it is published on `CRAN`.

`.gitignore`

GitHub sees every file in your repository as one of the following three:

- **tracked files** that have been (previously) staged (i.e., added for commit) and committed;
- **untracked files** that have not been staged or committed;
- **ignored files** that have been explicitly ignored.

It may be wise to instruct Git to ignore changes in some files. For example, compiled files (think about `.com`, `.exe`, `.o`, `.so`, etc), archives (e.g. `.zip`, `.tar`, `.rar`), logs (`.log`) and files generated in runtime (`.temp`) do not have to be tracked by Git. The same holds for hidden system files (e.g. `.DS_Store` or `Thumbs.db`). Adding such file types to a file named `.gitignore` and placing that file in the root of your repository will take care of focusing Git's energy on useful files only. For common `.gitignore` examples, see this repository. There are many examples inside, such as this `.gitignore` example for `R`. When you create a new repository on GitHub, you have the option of creating a default .gitignore file for your programming language of choice.

Suggested reading and supplementary material

Make sure to read the following works:

- Happy Git with R covers pretty much everything you need to marry Git and `R`
- Excuse me, do you have a moment to talk about version control? by Jenny Bryan

The following links may also be very useful:

- the Pro Git book and online resources.
- an old video walkthrough about Git and `Rstudio` from the 'Markup Languages & Reproducible Programming in Statistics' course.
- the GitHub Glossary for all terminology
- become a Git Guru

- this online Git book is a very good resource
- learn how to write gitmoji commit messages
- Getting Started with Git and GitHub at UU Geosciences
- UU library workshop on project-based workflows with Quarto: Writing Reproducible Manuscripts in R & Python

Installing Git

How to install a Git client? And which one to choose?

> GUI or terminal?
>
> Ultimately, you'll want to learn how to use Git through the command line interface (CLI). It offers better and more comprehensive functionality. Take this 15-minute course to get a gentle introduction. But do not be afraid that you miss out on the CLI if you don't study this link: in 'Markup Languages and Reproducible Programming in Statistics' we'll explore in detail how to handle Git when things go haywire. You'll be a CLI-wizard by then.

For now, let's stick with the GitHub Desktop GUI.

Installing on Mac

I suggest you install Git by downloading and installing `GitHub Desktop`. GitHub's desktop application is a nice GUI and, naturally, integrates well into the repository workflow on GitHub.

When installed, you can go to `GitHub Desktop > Install Command Line Tool`

After a reboot, all should be set.

Installing on Windows

Download and install `Git for Windows`, Then download and install `GitHub Desktop`. GitHub's desktop application is a nice GUI and, naturally, integrates well into the repository workflow on GitHub.

After a reboot, all should be set.

Linking GitHub and RStudio

Securely linking your local machine to the remote repository is vital when collaborating with other people. You would not want a potential hacker to have contributor access to any of your projects.

Use the instructions from Happy Git with R chapter 12 to Connect RStudio to Git and GitHub. If you want to, you could also watch an old walkthrough video, which details the process of linking GitHub to your machine and RStudio. Below we explain the rationale of using both an SSH key and a personal access token.

If you still experience problems after following the aforementioned walkthrough, check this chapter.

To learn more about maintaining an R package as GitHub repository within RStudio, have a look at this guide by Hadley Wickham.

SSH keys

With an SSH key you can identify yourself to an online server (in this case the GitHub server) without having to log in every time. It is like your machine having access to an online server through a unique biometric security measure, but instead of biometric data a bits-and-bytes *hash code* is communicated every time. You can use an SSH key to link RStudio to your GitHub repository, but it is better to use a personal access token for this.

Personal access tokens

Because GitHub requires Two-Factor Authentication (2FA), your username and password are not sufficient to `push commits` to GitHub through RStudio. To solve this, follow these steps on github.com as detailed in this walkthrough video:

1. Log in to your account
2. Click on your profile photo (upper right corner) and select `Settings`
3. Go to `Developer settings`
4. Select `Personal access tokens` in the left sidebar. The classic token is sufficient
5. Click `Generate new token`
6. Give the token a name
7. Select at least the `repo` scope; you'll need these permissions to access repositories. Also choose a relevant expiration: the longer the more convenient, the shorter the safer. You can always generate a new token if you need to.
8. Click Generate token

Copy the token. The token **will not be displayed again**, so paste it directly where it is needed. Remember that you can always generate a new token if you need to.

In RStudio, paste the generated token in the password field when RStudio asks for your credentials. The token will now serve as the unique authenticated link instead of your password.

Exercises

0. Install and set-up Git, GitHub, GitHub Desktop and RStudio (see above).
1. Fork the course repository. You can do this by clicking the `Fork` button on the top right of the repository page. This will create a copy of the repository on your own GitHub account.
2. Clone the fork to your machine. You can do this by clicking the green `Code` button and copying the URL. Instructions on the table below.
3. Create a new branch called `[yourname]`, where you replace [yourname] with your first name.
4. Update the file `story.qmd` by adding three words to the sentence and commit the changes. Write a commit message that explains why you made those changes. Instructions on the table below.
5. Push to your GitHub fork. Instructions on the table below; pick your own preferred interface.
6. Send a pull request (PR) to incorporate your changes into the `upstream/main` branch (i.e. `mlarpis/git2026@main`). You can do this by going to your fork on GitHub and clicking the `Pull request` button. Make sure to select the correct branches.
7. We will merge the PRs in the order they are presented in, as long as there are no conflicts.
8. If conflicts arise and your PR can no longer be merged, update it by pulling the changes from the remote, submitting another commit, and pushing it to your fork. This will update the PR automatically.

Instruction	GitHub Desktop	RStudio	Terminal
Clone the fork	Select `File > Clone repository`, choose the **URL** tab, paste the copied URL, choose a local path, and click `Clone`.	`File > New Project > Version Control > Git`. Paste the URL in the `Repository URL` field and choose a directory where you want to store the repository on your machine.	`git clone copied_url`*

Instruction	GitHub Desktop	RStudio	Terminal
Commit the changes	Open the **Changes** tab, tick the box next to `story.qmd`, enter a commit message in the box at the bottom left, and click `Commit to main`.	Click on the Git tab, select `story.qmd`, write the commit message and click `Commit`.	`git add story.qmd; git commit -m "commit_message"`**
Push to fork	Click **Push origin** in the top bar. This uploads your commit to your fork on GitHub.	Click the green `Push` button in the Git tab.	`git push`***

Note. *replace `copied_url` by the URL you copied **replace `commit_message` by the commit message ***this may trigger a warning; read it and follow the instructions.

Instructions for handing in your work

- This week, there is no hand-in via Brightspace. Instead, you hand in your work via GitHub, as described above.
- Submit your pull request at least one day before the next class session. Make sure to resolve any merge conflicts.